

Implementasi Security Orchestration, Automation, and Response (SOAR)
Menggunakan Stack Sumber Terbuka untuk Deteksi Malware dan Phishing
dengan Active Response Human-in-the-Loop



OLEH:

Ravi Arnan Irianto (2305551076)

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS UDAYANA
2026**

KATA PENGANTAR

Puji dan syukur penulis panjatkan kepada Tuhan Yang Maha Esa, atas segala berkat dan rahmat-Nya yang membantu penulis dalam menyelesaikan tugas akhir dengan judul **“Implementasi *Security Orchestration, Automation, and Response* (SOAR) Menggunakan *Stack Sumber Terbuka* untuk Deteksi Malware dan Phishing dengan *Active Response Human-in-the-Loop*”** ini. Dalam penyelesaian tugas akhir ini, penulis juga mendapatkan banyak bantuan materil dan non materil dari berbagai pihak. Oleh karena itu, penulis ingin mengucapkan terima kasih kepada:

1. Bapak Ir. I Nyoman Piarsa, ST., MT., IPM selaku dosen pembimbing I yang telah membimbing penulis selama penelitian dan penulisan tugas akhir ini, termasuk masukan agar mekanisme *Active Response* dirancang secara *human-in-the-loop* dan tidak melakukan isolasi otomatis secara buta.
2. Orang tua dan keluarga yang telah memberikan dukungan penuh kepada penulis selama menempuh perkuliahan.
3. Teman-teman penulis, baik di lingkungan perkuliahan maupun di luar lingkungan perkuliahan yang telah memberikan bantuan dan dukungan kepada penulis selama menyusun tugas akhir.

Penulis menyadari bahwa tugas akhir ini masih jauh dari kata sempurna. Sehingga, penulis meminta maaf atas segala kekurangan dan berharap dengan pengalaman ini bisa membuat penulis lebih baik ke depannya.

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI.....	iii
DAFTAR GAMBAR.....	v
DAFTAR TABEL.....	vi
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Rumusan Masalah.....	3
1.3 Tujuan Penelitian.....	4
1.4 Manfaat Penelitian.....	4
1.5 Batasan Masalah.....	5
1.6 Luaran yang Diharapkan.....	6
1.7 Sistematika Penulisan.....	6
BAB II TINJAUAN PUSTAKA.....	8
2.1 State of The Art.....	8
2.2 Konsep Keamanan Siber.....	10
2.3 Intrusion Detection and Prevention System (IDS/IPS).....	10
2.4 Security Information and Event Management (SIEM).....	11
2.5 Wazuh sebagai Platform SIEM dan HIDS.....	11
2.6 Security Orchestration, Automation, and Response (SOAR).....	12
2.7 Deteksi Malware dan File Integrity Monitoring.....	12
2.8 Deteksi Phishing.....	13
2.9 Orkestrasi Workflow dengan n8n.....	13
2.10 Threat Intelligence dan Analisis AI Lokal.....	14
2.11 Active Response Human-in-the-Loop dan Integrasi Telegram Bot.....	14
BAB III METODOLOGI PENELITIAN.....	16
3.1 Waktu dan Tempat Penelitian.....	16
3.2 Sumber Data.....	16
3.3 Instrumen Perancangan Sistem.....	17
3.3.1 Perangkat Keras (Hardware).....	17
3.3.2 Perangkat Lunak (Software).....	17

3.4 Tahapan Penelitian.....	18
3.5 Metode Pengembangan Sistem.....	21
3.5.1 Analisis Kebutuhan (Requirement Analysis).....	22
3.5.2 Desain Sistem (System Design).....	23
3.5.3 Implementasi (Implementation).....	23
3.5.4 Pengujian (Testing).....	24
3.5.5 Pemeliharaan (Maintenance).....	25
3.6 Desain Arsitektur Sistem.....	25
3.7 Perancangan Pipeline Deteksi dan Klasifikasi Severity.....	27
3.8 Desain Active Response Human-in-the-Loop dan Integrasi Telegram Bot.....	30
DAFTAR PUSTAKA.....	35

DAFTAR GAMBAR

Gambar 1.1 Tren Insiden Siber Berbasis Malware dan Phishing.....	3
Gambar 3.1 Tahapan Penelitian.....	19
Gambar 3.2 Tahapan Model Waterfall.....	22
Gambar 3.3 Desain Arsitektur Sistem SOAR.....	26
Gambar 3.4 Alur Pipeline Deteksi End-to-End.....	28
Gambar 3.5 Sequence Active Response Human-in-the-Loop.....	31
Gambar 3.6 Desain Integrasi Telegram Bot.....	33

DAFTAR TABEL

Tabel 3.1 Spesifikasi Perangkat Keras.....	17
Tabel 3.2 Daftar Perangkat Lunak.....	17
Tabel 3.3 Aturan Klasifikasi Severity.....	29
Tabel 3.4 Pemetaan Lokasi FIM terhadap MITRE ATT&CK.....	30
Tabel 3.5 Tabel Indikator Pengujian.....	24

BAB I

PENDAHULUAN

Bab I memuat pendahuluan dari penelitian “Implementasi *Security Orchestration, Automation, and Response* (SOAR) Menggunakan *Stack* Sumber Terbuka untuk Deteksi Malware dan Phishing dengan *Active Response Human-in-the-Loop*”. Bagian pendahuluan membahas mengenai latar belakang, rumusan masalah, tujuan penelitian, manfaat penelitian, batasan masalah, luaran yang diharapkan, dan sistematika penulisan dari penelitian ini. Berikut adalah pembahasannya.

1.1 Latar Belakang

Perkembangan teknologi informasi dan komunikasi yang pesat telah mendorong berbagai sektor untuk beralih ke sistem digital. Aktivitas seperti transaksi keuangan, komunikasi data, hingga pengelolaan layanan publik kini semakin bergantung pada jaringan komputer. Namun, peningkatan konektivitas ini juga diikuti oleh meningkatnya ancaman siber yang semakin kompleks dan sulit dideteksi. Serangan seperti *malware injection*, *ransomware*, dan *phishing* dapat menyebabkan gangguan layanan, pencurian data, hingga kerugian operasional yang besar bagi organisasi.

Dua kategori ancaman yang paling banyak dialami oleh organisasi skala kecil hingga menengah adalah *malware* dan *phishing*. *Malware* berupa berkas berbahaya yang menyusup ke *endpoint*, sering kali melalui unduhan dari internet atau lampiran surel, dapat menjadi titik awal kompromi sistem. Sementara itu, *phishing* yang mengarahkan pengguna ke tautan berbahaya menjadi vektor serangan rekayasa sosial yang paling umum untuk pencurian kredensial. Kedua jenis ancaman ini berkembang sangat cepat sehingga deteksi saja tidak lagi cukup; organisasi membutuhkan kemampuan untuk **merespons** insiden secara cepat dan konsisten.

Sistem keamanan jaringan tradisional umumnya hanya berhenti pada tahap deteksi dan notifikasi. *Security Information and Event Management* (SIEM) seperti Wazuh mampu mengumpulkan serta menganalisis *log* dari berbagai *endpoint* dan menghasilkan *alert*, namun proses analisis lanjutan (pengayaan/*enrichment*), pengambilan keputusan, dan eksekusi respons sering kali masih dikerjakan secara manual oleh tim *Security Operations Center* (SOC). Pendekatan manual ini memiliki keterbatasan: rentan terhadap kelelahan analis (*alert fatigue*),

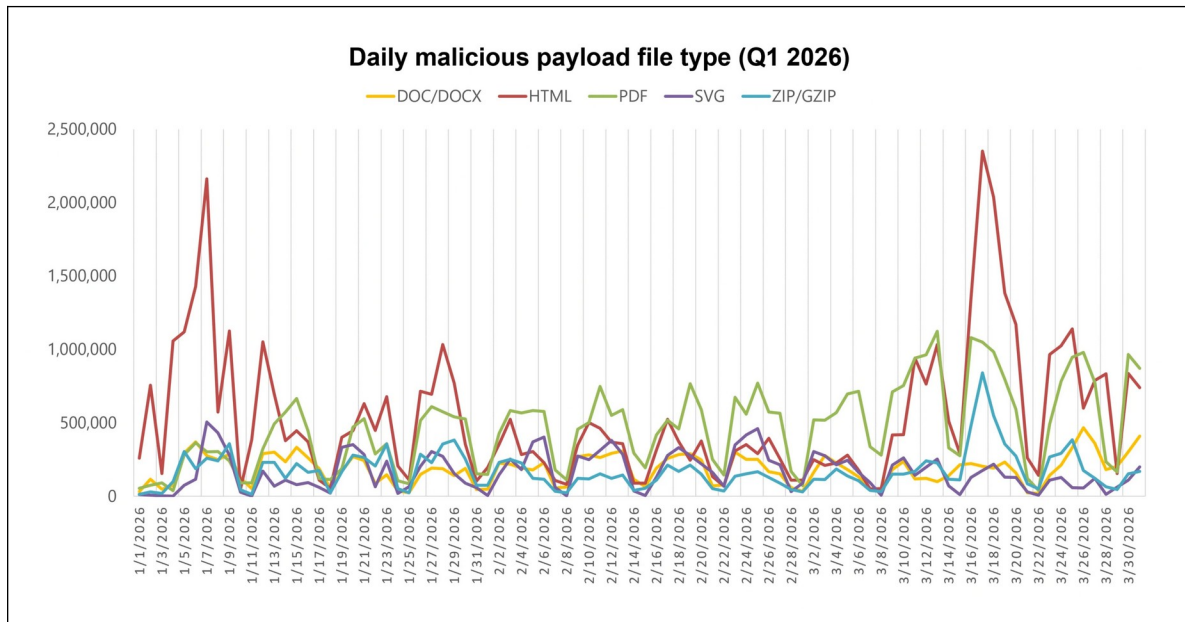
lambat dalam merespons, serta tidak konsisten. Akibatnya, jeda waktu antara terdeteksinya ancaman dan tindakan penanggulangan (*containment*) menjadi panjang, padahal pada serangan modern jeda inilah yang menentukan besar kecilnya dampak.

Untuk menjembatani kesenjangan antara *deteksi* dan *respons*, muncul paradigma **SOAR** (*Security Orchestration, Automation, and Response*). SOAR mengintegrasikan berbagai *tool* keamanan ke dalam satu alur kerja terotomasi (*playbook*) yang melakukan pengumpulan *alert*, pengayaan dengan *threat intelligence*, klasifikasi tingkat keparahan, hingga eksekusi tindakan respons. Platform SOAR komersial seperti Cortex XSOAR atau Splunk SOAR menawarkan kemampuan ini, tetapi memiliki biaya lisensi yang tinggi dan tidak selalu terjangkau bagi institusi pendidikan maupun organisasi kecil-menengah. Oleh karena itu, dibutuhkan sebuah implementasi SOAR yang efektif namun dibangun sepenuhnya di atas perangkat **sumber terbuka** (*open-source*).

Di sisi lain, otomasi respons yang sepenuhnya buta (*fully automated active response*) juga menyimpan risiko. Tindakan agresif seperti memblokir IP, mematikan layanan, atau mengisolasi berkas yang dipicu otomatis oleh setiap *alert* berpotensi menimbulkan gangguan operasional ketika *alert* tersebut ternyata *false positive*. Untuk *alert* kritis, keputusan untuk melakukan tindakan destruktif sebaiknya tetap melibatkan pertimbangan manusia. Konsep inilah yang disebut **Active Response human-in-the-loop**, yaitu otomasi menyiapkan seluruh konteks dan opsi tindakan, tetapi eksekusi tindakan kritis baru dijalankan setelah disetujui oleh analis.

Berdasarkan permasalahan tersebut, penelitian ini mengusulkan pembangunan sebuah sistem SOAR berbasis *stack* sumber terbuka yang memadukan **Wazuh** sebagai SIEM dan *Host-based Intrusion Detection System* (HIDS), **n8n** sebagai mesin orkestrasi *workflow*, **VirusTotal** sebagai sumber *threat intelligence* eksternal, **Ollama** (model LLM lokal) sebagai modul analisis kontekstual, dan **Telegram Bot** sebagai kanal notifikasi sekaligus antarmuka pengambilan keputusan dua arah. Sistem difokuskan pada dua jenis ancaman, yaitu deteksi *malware* (melalui *File Integrity Monitoring*) dan *phishing* (melalui analisis *log* akses tautan), dengan mekanisme *Active Response human-in-the-loop*: untuk *alert* kritis, analis memutuskan melalui tombol pada notifikasi Telegram apakah berkas akan diisolasi (dikarantina) atau dibiarkan karena dianggap *false positive*.

Gambar 1.1 menggambarkan tren peningkatan insiden siber berbasis *malware* dan *phishing*. Tren ini memperkuat kebutuhan akan sistem yang tidak hanya mendeteksi, tetapi juga mampu mengorkestrasi respons secara cepat, terukur, dan tetap terkendali oleh manusia.



Gambar 1.1 Tren Insiden Siber Berbasis Malware dan Phishing

Melalui kombinasi Wazuh sebagai sistem deteksi, n8n sebagai mesin orkestrasi, VirusTotal dan Ollama sebagai lapisan pengayaan, serta Telegram Bot sebagai kanal respons interaktif, diharapkan sistem ini mampu memberikan solusi pertahanan siber yang lebih adaptif, efisien, dan responsif tanpa kehilangan kontrol manusia atas tindakan kritis. Dengan adanya sistem SOAR sumber terbuka ini, administrator jaringan dapat lebih cepat dalam mengidentifikasi serta menangani ancaman, sehingga keamanan dan stabilitas sistem dapat terjaga dengan lebih baik dan biaya implementasi tetap rendah.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang sudah diuraikan dapat dirumuskan beberapa rumusan masalah. Rumusan masalah dari penelitian ini adalah sebagai berikut.

- a. Bagaimana merancang dan mengimplementasikan sistem SOAR berbasis *stack* sumber terbuka (Wazuh, n8n, VirusTotal, Ollama, dan Telegram Bot) untuk mendeteksi *malware* dan *phishing*?

- b. Bagaimana mengorkestrasi alur kerja deteksi mulai dari pengumpulan *alert* Wazuh, pengayaan dengan *threat intelligence* dan analisis AI lokal, hingga klasifikasi tingkat keparahan (*severity*) secara otomatis dan *real-time*?
- c. Bagaimana merancang mekanisme *Active Response human-in-the-loop* sehingga tindakan kritis seperti isolasi berkas hanya dieksekusi setelah mendapat persetujuan analis melalui Telegram, dan sejauh mana sistem yang dikembangkan mampu memberikan solusi pertahanan siber yang lebih adaptif, efisien, dan responsif dibandingkan dengan pendekatan deteksi-notifikasi manual?

1.3 Tujuan Penelitian

Sesuai dengan rumusan masalah yang telah diuraikan, dapat disimpulkan tujuan dari penelitian ini adalah sebagai berikut.

- a. Merancang dan mengimplementasikan sistem SOAR berbasis *stack* sumber terbuka yang memadukan Wazuh, n8n, VirusTotal, Ollama, dan Telegram Bot untuk mendeteksi *malware* dan *phishing*.
- b. Mengorkestrasi alur kerja deteksi otomatis mulai dari pengumpulan *alert*, pengayaan *threat intelligence* dan analisis AI lokal, hingga klasifikasi *severity* secara *real-time* dan mudah dipantau.
- c. Mengembangkan mekanisme *Active Response human-in-the-loop* untuk eksekusi tindakan kritis (isolasi berkas) yang aman dan terkendali, serta mengetahui sejauh mana sistem yang dikembangkan mampu memberikan solusi pertahanan siber yang lebih adaptif, efisien, dan responsif dibandingkan pendekatan deteksi-notifikasi manual.

1.4 Manfaat Penelitian

Berdasarkan tujuan dari penelitian ini, manfaat-manfaat yang didapatkan dapat diuraikan sebagai berikut.

a. Manfaat Akademik

- Menambah wawasan dan referensi ilmiah dalam bidang keamanan jaringan, khususnya pada penerapan konsep SOAR (*Security Orchestration, Automation, and Response*) menggunakan perangkat sumber terbuka.

- Menjadi dasar penelitian lanjutan dalam pengembangan *security playbook*, orkestrasi respons insiden, serta integrasi *threat intelligence* dan model bahasa lokal (LLM) pada konteks keamanan siber.
 - Memberikan kontribusi bagi mahasiswa atau peneliti dalam memahami integrasi SIEM dengan mesin orkestrasi dan automasi respons yang tetap memperhatikan kendali manusia (*human-in-the-loop*).
- b. **Manfaat Teknis**
- Menghasilkan sistem pertahanan siber yang mampu mendeteksi *malware* dan *phishing* sekaligus mengorkestrasi respons secara otomatis dengan pengayaan *threat intelligence* dan analisis AI lokal.
 - Menyediakan mekanisme notifikasi dan tindakan respons interaktif melalui Telegram Bot, sehingga proses deteksi dan respons dapat berjalan cepat, konsisten, dan tetap terkendali.
- c. **Manfaat Praktis**
- Dapat diterapkan di lingkungan kampus maupun organisasi kecil hingga menengah untuk meningkatkan keamanan infrastruktur digital dengan biaya rendah karena seluruhnya berbasis perangkat sumber terbuka.
 - Berpotensi menjadi dasar pengembangan produk keamanan siber mandiri (*SOC-in-a-box*) yang dapat diimplementasikan secara luas di institusi pendidikan maupun industri.

1.5 Batasan Masalah

Sebuah penelitian dibuat dengan berfokus pada suatu permasalahan serta solusi yang dapat diberikan. Adapun batasan atau fokus dari penelitian ini adalah sebagai berikut.

- Penelitian hanya berfokus pada pengembangan sistem keamanan pada lingkungan *endpoint* skala laboratorium/SMB, bukan pada infrastruktur jaringan berskala besar atau jaringan publik yang kompleks.
- Jenis ancaman yang dideteksi dibatasi pada **deteksi *malware*** (melalui *File Integrity Monitoring* dan reputasi *hash*) dan **deteksi *phishing*** (melalui analisis *log akses*

tautan/URL). Serangan berbasis jaringan intensif seperti DDoS, serta serangan *fileless/memory-only*, tidak menjadi fokus utama.

- c. Tindakan *Active Response* yang diimplementasikan dibatasi pada **isolasi/karantina berkas** (*quarantine-file*) yang dieksekusi dengan persetujuan analis (*human-in-the-loop*). Tindakan respons lain seperti pemblokiran IP otomatis (*firewall-drop*) sengaja tidak diaktifkan untuk menghindari isolasi buta atas *false positive*.
- d. Pengayaan *threat intelligence* menggunakan VirusTotal API pada *free tier* dengan keterbatasan kuota (4 permintaan/menit, 500/hari), dan analisis kontekstual menggunakan model LLM lokal llama3.2:3b melalui Ollama.
- e. Evaluasi sistem difokuskan pada pengujian kemampuan deteksi, kecepatan pipeline (*Mean Time to Detect* dan *Mean Time to Respond*), keberhasilan orkestrasi antarkomponen, serta keberhasilan eksekusi *Active Response* setelah persetujuan analis. Aspek ketersediaan tinggi (HA), skalabilitas untuk ribuan *agent*, dan performa SIEM untuk beban *log* sangat besar tidak dibahas secara mendalam.

1.6 Luaran yang Diharapkan

Penelitian ini diharapkan dapat menghasilkan berupa sistem SOAR sumber terbuka yang menggabungkan Wazuh sebagai SIEM/HIDS, n8n sebagai mesin orkestrasi *workflow*, VirusTotal dan Ollama sebagai lapisan pengayaan, serta terhubung otomatis dengan Telegram Bot sebagai kanal notifikasi dan pengambilan keputusan. Sistem mampu mendeteksi *malware* dan *phishing*, mengklasifikasikan tingkat keparahan secara *real-time*, dan menjalankan *Active Response* berupa isolasi berkas setelah disetujui analis (*human-in-the-loop*), serta menyediakan jejak audit (*audit trail*) lengkap dari setiap insiden.

1.7 Sistematika Penulisan

Dalam penelitian ini, agar pembahasan menjadi terfokus, maka dibuatkan sistematika penulisan penelitian sebagai berikut.

1. **BAB I: PENDAHULUAN** Memaparkan tentang hal-hal yang melatarbelakangi penelitian ini di latar belakang, dirumuskan masalahnya dalam rumusan masalah, dan dibatasi pada batasan masalah, tujuan, manfaat, luaran, dan sistematika penulisan.

2. **BAB II: TINJAUAN PUSTAKA** Mengulas tentang landasan teori yang digunakan dalam perancangan proyek ini, yang didapat dari berbagai referensi yang ada, meliputi konsep keamanan siber, SIEM, Wazuh, SOAR, deteksi malware dan phishing, orkestrasi n8n, serta Active Response human-in-the-loop.
3. **BAB III: METODE PENELITIAN** Membahas tentang metode yang digunakan dalam perancangan sistem dan alur sistem, mencakup tahapan penelitian, metode pengembangan, desain arsitektur, perancangan pipeline deteksi, serta desain integrasi Telegram dan Active Response.
4. **BAB IV: HASIL DAN PEMBAHASAN** Membahas tentang hasil implementasi dari sistem yang telah dibuat beserta hasil pengujiannya.
5. **BAB V: PENUTUP** Berisi kesimpulan beserta saran untuk pengembangan yang bisa dilakukan lebih lanjut.

BAB II TINJAUAN PUSTAKA

2.1 State of The Art

Penelitian yang dilakukan oleh Farhan dkk. mengenai penerapan Wazuh sebagai SIEM dengan fitur *Active Response* dan integrasi Telegram API untuk mendeteksi serta menanggulangi serangan *brute force* pada sistem informasi GT-I2TI Universitas Trisakti. Penelitian ini menggunakan metodologi *Security Lifecycle Model* yang terdiri dari empat tahap utama: *Identify*, *Assess*, *Protect*, dan *Monitor*. Wazuh dikonfigurasi untuk mendeteksi percobaan login gagal pada berbagai protokol seperti HTTP, SSH, HTTPS, FTP, IMAP, dan RDP, sementara fitur *Active Response* digunakan untuk secara otomatis memblokir alamat IP penyerang. Integrasi dengan Telegram Bot API memungkinkan pengiriman notifikasi *real-time* kepada administrator. Hasil pengujian menunjukkan Wazuh mampu mendeteksi 100% serangan *brute force* dengan waktu rata-rata deteksi sebesar 80,51 detik untuk interval 1 detik, dan waktu respons aktif (*Active Response Reaction Time*) rata-rata 0,51 detik (Farrel et al., 2024). Penelitian ini menjadi acuan penting, namun *Active Response*-nya sepenuhnya otomatis; penelitian saat ini berbeda dengan menempatkan keputusan eksekusi pada analisis (*human-in-the-loop*).

Penelitian yang dilakukan oleh Faruq dkk. mengenai penerapan Wazuh sebagai sistem *Security Information and Event Management* (SIEM) yang diintegrasikan dengan Telegram Bot untuk melakukan deteksi, analisis, dan visualisasi *log* keamanan secara *real-time*. Studi kasus dilakukan di lingkungan Pesantren Teknologi Informasi dan Komunikasi (PeTIK) Jombang dengan tujuan meningkatkan efisiensi *monitoring* dan respons terhadap ancaman seperti *Brute Force*, *DoS Attack (SYN Flood)*, dan *SQL Injection*. Implementasi Wazuh berhasil menampilkan *log* insiden melalui *dashboard* serta mengirimkan notifikasi otomatis ke Telegram Bot, meskipun masih terdapat keterbatasan pada beberapa jenis serangan yang tidak menghasilkan notifikasi penuh (F. A. Saputra et al., 2024).

Penelitian yang dilakukan oleh Gede Parama Andika mengenai efektivitas Wazuh sebagai sistem deteksi intrusi (IDS) untuk meningkatkan keamanan jaringan web di lingkungan rumah sakit. Penelitian ini berfokus pada identifikasi dan mitigasi tiga jenis serangan utama, yaitu *Local File Inclusion (LFI)*, *XML External Entity (XXE)*, dan *Denial of Service (DoS)*,

dengan memanfaatkan Wazuh *Dashboard* untuk mendeteksi serta mengirimkan notifikasi otomatis melalui Discord. Pengujian dilakukan menggunakan simulasi berbasis Docker di server AWS, di mana setiap serangan berhasil terdeteksi oleh Wazuh dengan pemberitahuan *real-time* (Gede Parama Antara & Ika Dyah Agustia Rachmawati, 2024).

Penelitian yang dilakukan oleh Ariyanto dkk. mengenai pengembangan sistem deteksi dan tanggapan ancaman siber otomatis berbasis Wazuh SIEM, yang terintegrasi dengan Fail2Ban untuk pemblokiran IP dinamis serta Telegram API sebagai kanal notifikasi *real-time*. Sistem ini dirancang menggunakan metodologi PPDIIOO (*Prepare, Plan, Design, Implement, Operate, Optimize*) dan dilengkapi modul klasifikasi berbasis *machine learning* (Random Forest) untuk mengidentifikasi pola serangan *Brute Force* dan *DDoS*, menghasilkan akurasi deteksi 98,4% dan waktu tanggap di bawah 10 detik (Ariyanto et al., 2025). Penelitian ini menunjukkan kematangan pendekatan otomasi penuh, sementara penelitian saat ini menyoroti pentingnya kendali manusia pada tindakan kritis.

Penelitian yang dilakukan oleh Dwi Prasetyo dkk. mengenai evaluasi kinerja sistem deteksi intrusi berbasis *host* (HIDS) Wazuh dalam menghadapi serangan *Brute Force* dan *Denial of Service* (DoS). Penelitian ini menggunakan skenario pengujian dengan alat serangan Medusa dan Hydra untuk *Brute Force* serta Hping3 dan Metasploit untuk *DoS Attack*. Hasil uji menunjukkan bahwa Wazuh berhasil mendeteksi serangan *Brute Force* secara *real-time* melalui tampilan *alert evolution* dan skema MITRE ATT&CK, namun gagal mendeteksi serangan DoS secara akurat, Wazuh hanya menampilkan aktivitas *listened port* tanpa memberikan *alert* bahaya. Temuan ini menegaskan bahwa meskipun Wazuh efektif sebagai detektor aktivitas login mencurigakan, kemampuan deteksi terhadap serangan berbasis trafik intensif seperti DoS masih terbatas (Dwi Prasetyo et al., 2023).

Penelitian yang dilakukan oleh Delvita dkk. mengenai pengembangan sistem keamanan berbasis web bernama *SecurityShield* yang mampu mendeteksi aktivitas anomali secara *real-time* melalui analisis *log* aktivitas pengguna. Sistem ini dilengkapi autentikasi dua faktor (OTP Telegram), *dashboard* interaktif berbasis WebSocket, serta notifikasi otomatis untuk administrator ketika terdeteksi aktivitas mencurigakan (Delvita aulia artika et al., 2025). Penelitian ini relevan dalam hal pemanfaatan Telegram sebagai kanal komunikasi keamanan dua arah.

Berdasarkan kajian *state of the art* di atas, sebagian besar penelitian sebelumnya berhenti pada otomasi penuh (*auto active response*) atau hanya pada notifikasi satu arah. Penelitian ini memberikan kebaruan (*novelty*) dengan: (1) mengintegrasikan **orkestrasi SOAR penuh** menggunakan n8n sebagai mesin *playbook* yang menyatukan Wazuh, VirusTotal, dan LLM lokal; (2) menambahkan **analisis kontekstual berbasis AI lokal** (Ollama) yang menjaga kedaulatan data; dan (3) menerapkan model **Active Response *human-in-the-loop*** di mana isolasi berkas hanya dieksekusi setelah disetujui analisis melalui tombol Telegram, sehingga menghindari isolasi buta atas *false positive*.

2.2 Konsep Keamanan Siber

Keamanan siber merupakan upaya melindungi sistem komputer, data, dan jaringan dari ancaman digital. Menurut NIST, keamanan siber mencakup teknik, kebijakan, dan praktik untuk mencegah pencurian, kerusakan, serta gangguan informasi digital (Puteri et al., 2024). Dasarnya berlandaskan CIA Triad yaitu *Confidentiality*, *Integrity*, dan *Availability* sebagai standar utama perlindungan data. Strategi umum yang digunakan meliputi *firewall*, enkripsi, autentikasi ganda, dan pelatihan kesadaran keamanan. Ancaman yang sering terjadi antara lain DDoS, *ransomware*, *spyware*, *malware*, *phishing*, serta *man-in-the-middle attack* yang dapat mengganggu layanan jaringan dan membahayakan data pengguna. Dalam konteks penelitian ini, fokus diberikan pada perlindungan terhadap integritas berkas (*Integrity*) dari *malware* dan perlindungan pengguna dari rekayasa sosial *phishing*.

2.3 Intrusion Detection and Prevention System (IDS/IPS)

Intrusion Detection System (IDS) adalah sistem yang memantau lalu lintas jaringan atau aktivitas sistem untuk mendeteksi perilaku mencurigakan lalu mengirimkan notifikasi kepada administrator saat terjadinya indikasi sebuah aktivitas mencurigakan yang terdeteksi. *Intrusion Prevention System* (IPS) bertugas dengan cara yang sama dengan IDS dan dapat mencegah aktivitas tersebut secara otomatis. IDS dibagi menjadi dua jenis, yaitu:

1. *Signature-based Detection*, yang mengidentifikasi serangan berdasarkan pola yang telah dikenal (*known attack signatures*).
2. *Anomaly-based Detection*, yang mendeteksi aktivitas tak biasa dengan membandingkannya terhadap pola perilaku normal sistem.

Sementara IPS memperluas fungsi IDS dengan memblokir, memutus koneksi, atau mengubah konfigurasi jaringan secara *real-time* untuk mengurangi dampak serangan. Pada konteks penelitian ini, Wazuh berperan sebagai *Host-based IDS* (HIDS), dan kemampuan IPS direpresentasikan oleh fitur *Active Response*-nya, yang dalam penelitian ini sengaja dijalankan secara *human-in-the-loop* alih-alih pencegahan otomatis penuh.

2.4 Security Information and Event Management (SIEM)

Security Information and Event Management (SIEM) adalah sistem keamanan terpusat yang menggabungkan dua fungsi utama: *Security Information Management* (SIM) dan *Security Event Management* (SEM). SIEM berperan dalam mengumpulkan *log* dari berbagai sumber (sistem operasi, *firewall*, server, *endpoint*, dan aplikasi), menormalisasi data tersebut, lalu menganalisisnya untuk mengidentifikasi pola anomali dan insiden keamanan (Dyan Heluka & Sulistyono, n.d.). SIEM modern tidak hanya berfungsi sebagai pusat *monitoring log*, tetapi juga mendukung *threat intelligence*, *correlation rules*, *machine learning*, dan *real-time alerting* untuk meningkatkan efektivitas deteksi ancaman. Dengan kemampuan ini, SIEM menjadi tulang punggung sistem pertahanan siber organisasi dalam mengelola volume *log* yang besar secara efisien dan responsif. Namun, SIEM pada dirinya sendiri umumnya berhenti pada deteksi dan korelasi; kemampuan untuk **mengorkestrasi** dan **mengotomasi respons** lintas *tool* inilah yang dilengkapi oleh lapisan SOAR.

2.5 Wazuh sebagai Platform SIEM dan HIDS

Wazuh adalah platform keamanan sumber terbuka yang berfungsi sebagai SIEM dan *Host-based Intrusion Detection System* (HIDS). Wazuh terdiri dari tiga komponen utama:

1. **Wazuh Agent**, yang bertugas mengumpulkan *log* dan data aktivitas dari *endpoint*, termasuk *File Integrity Monitoring* (FIM), *log collection*, dan *configuration assessment*.
2. **Wazuh Manager**, yang menganalisis data *log* berdasarkan aturan (*ruleset*) dan mendeteksi potensi ancaman, serta memicu *integration* dan *Active Response*.
3. **Wazuh Indexer dan Dashboard** (OpenSearch/Kibana), yang menyimpan, mengindeks, dan menampilkan hasil *monitoring*, notifikasi, dan laporan keamanan dalam antarmuka visual.

Wazuh juga menyediakan modul *File Integrity Monitoring*, *Vulnerability Detection*, dan *Active Response*. Keunggulan Wazuh terletak pada kemampuannya beroperasi efisien dengan

sumber daya rendah dibanding platform SIEM lain seperti Splunk atau QRadar, menjadikannya pilihan populer untuk implementasi sistem keamanan skala menengah dan penelitian akademik. Modul yang paling relevan untuk penelitian ini adalah **FIM** (untuk deteksi *malware* berbasis perubahan berkas), **integrator** (untuk menyalurkan *alert* ke mesin orkestrasi), dan **Active Response** (untuk eksekusi tindakan quarantine-file melalui Wazuh API).

2.6 Security Orchestration, Automation, and Response (SOAR)

Security Orchestration, Automation, and Response (SOAR) adalah kategori solusi keamanan yang dirancang untuk mengoordinasikan, menjalankan, dan mengotomasi tugas-tugas penanganan insiden antarberbagai *tool* dan SDM. Istilah ini dipopulerkan oleh Gartner dan mencakup tiga kapabilitas utama:

1. **Orchestration (Orkestrasi)**: menyatukan berbagai *tool* keamanan (SIEM, *threat intelligence*, *endpoint*, ticketing) ke dalam satu alur kerja terkoordinasi.
2. **Automation (Automasi)**: menjalankan tugas berulang secara otomatis melalui *playbook*, seperti pengayaan *alert*, pencarian reputasi *hash/URL*, dan klasifikasi *severity*.
3. **Response (Respons)**: mengeksekusi tindakan penanggulangan, baik secara otomatis maupun melalui persetujuan analis (*human-in-the-loop*).

Berbeda dengan SIEM yang berfokus pada *deteksi*, SOAR berfokus pada *aksi* setelah deteksi. SOAR mengurangi *Mean Time to Respond* (MTTR) dengan menghilangkan langkah-langkah manual yang berulang, sekaligus menjaga konsistensi penanganan insiden melalui *playbook* terstandarisasi. Pada penelitian ini, peran “otak” SOAR dijalankan oleh **n8n** sebagai mesin *playbook* yang mengorkestrasi Wazuh (deteksi), VirusTotal (intel), Ollama (analisis), dan Telegram (notifikasi + keputusan). Pendekatan ini membuktikan bahwa kapabilitas SOAR yang biasanya hanya tersedia pada platform komersial mahal dapat direplikasi menggunakan perangkat sumber terbuka.

2.7 Deteksi Malware dan File Integrity Monitoring

Malware (*malicious software*) adalah perangkat lunak yang dirancang untuk merusak, mengganggu, atau memperoleh akses tidak sah ke sistem. Salah satu pendekatan deteksi *malware* pada *endpoint* adalah melalui **File Integrity Monitoring** (FIM), yaitu pemantauan perubahan pada berkas dan direktori sistem secara *real-time*. Ketika sebuah berkas baru dibuat

atau dimodifikasi, FIM menghitung nilai *hash* kriptografis (SHA-256, SHA-1, MD5) dan membandingkannya terhadap *baseline*. Nilai *hash* ini kemudian dapat diperiksa terhadap basis data reputasi seperti VirusTotal untuk menentukan apakah berkas tersebut berbahaya.

Pada penelitian ini, Wazuh agent memantau lokasi-lokasi bernilai tinggi (*high-value paths*) yang relevan dengan model ancaman, bukan pemantauan menyeluruh yang menimbulkan *noise*. Pemilihan lokasi pemantauan dipetakan terhadap kerangka **MITRE ATT&CK** untuk memastikan cakupan terhadap teknik-teknik serangan yang umum, seperti *persistence* melalui cron/systemd, manipulasi kunci SSH, penggantian *binary*, dan akses awal melalui *phishing*.

2.8 Deteksi Phishing

Phishing adalah teknik rekayasa sosial yang berupaya menipu pengguna untuk mengakses tautan berbahaya atau memberikan informasi sensitif. Deteksi *phishing* pada penelitian ini dilakukan melalui analisis *log* akses tautan/URL: ketika sebuah URL mencurigakan terdeteksi (melalui aturan kustom Wazuh atau integrasi *proxy log*), URL tersebut diteruskan ke *pipeline* orkestrasi untuk diperiksa reputasinya terhadap VirusTotal. Berbeda dengan deteksi *malware* yang berbasis *hash* berkas, deteksi *phishing* berbasis reputasi URL dan domain. Pada implementasi, aturan kustom Wazuh dengan *rule_id* tertentu (mis. 87105, 87106, 100002) memicu integrasi ke *webhook* khusus *phishing* yang terpisah dari alur *malware*.

2.9 Orkestrasi Workflow dengan n8n

n8n adalah platform otomasi alur kerja (*workflow automation*) sumber terbuka yang memungkinkan pembuatan *pipeline* secara visual melalui rangkaian *node*. Setiap *node* merepresentasikan satu langkah, seperti menerima *webhook*, memanggil API eksternal, menjalankan kode JavaScript kustom, atau mengirim pesan. n8n dipilih sebagai mesin orkestrasi SOAR karena bersifat sumber terbuka (tanpa *vendor lock-in*), memiliki lebih dari 200 integrasi siap pakai, mendukung *node* kode kustom untuk logika lanjutan, serta menyediakan *webhook trigger* native yang cocok untuk menerima *alert* dari Wazuh integratord. Dengan n8n, *playbook* penanganan insiden dapat dirancang, dimodifikasi, dan dipantau eksekusinya secara visual, menjadikannya alternatif sumber terbuka terhadap platform SOAR komersial seperti Cortex XSOAR atau Splunk SOAR.

2.10 Threat Intelligence dan Analisis AI Lokal

Threat Intelligence adalah informasi terkait ancaman yang membantu mengkontekstualisasikan sebuah *alert*. Pada penelitian ini, **VirusTotal API** digunakan sebagai sumber *threat intelligence* eksternal untuk memeriksa reputasi *hash* berkas dan URL terhadap lebih dari 60 mesin antivirus. Hasil berupa statistik deteksi (mis. malicious: 65/67) menjadi salah satu masukan utama dalam klasifikasi *severity*.

Selain intel eksternal, sistem juga memanfaatkan **analisis AI lokal** menggunakan **Ollama** dengan model llama3.2:3b. Model bahasa (*Large Language Model/LLM*) ini menghasilkan analisis kontekstual yang mudah dibaca dalam Bahasa Indonesia, dengan *prompt* yang menyesuaikan tingkat keparahan (*severity-aware*). Keunggulan utama menjalankan LLM secara lokal adalah **kedaulatan data** (*data sovereignty*), data *alert* yang bersifat sensitif tidak keluar dari infrastruktur organisasi, serta tidak adanya biaya API yang cocok untuk *monitoring* berkelanjutan. *Trade-off*-nya adalah kualitas yang lebih rendah dibanding LLM komersial dan latensi inferensi berbasis CPU, yang masih dapat diterima untuk analisis singkat 2-3 kalimat.

2.11 Active Response Human-in-the-Loop dan Integrasi Telegram Bot

Integrasi sistem deteksi dengan platform komunikasi seperti Telegram Bot merupakan strategi modern untuk meningkatkan responsivitas dan efisiensi penanganan insiden keamanan. Penggunaan Telegram Bot memberikan solusi notifikasi *real-time* yang ringan, cepat, dan mudah diimplementasikan melalui Telegram Bot API. Dalam penelitian ini, Telegram Bot berfungsi untuk:

1. Mengirimkan notifikasi otomatis kepada administrator ketika sistem mendeteksi *malware* atau *phishing*, lengkap dengan hasil pengayaan VirusTotal dan analisis AI lokal.
2. Memberikan kemampuan tanggapan interaktif dua arah melalui *inline keyboard*, analisis dapat menyetujui atau menolak tindakan respons langsung dari notifikasi.
3. Menyediakan catatan aktivitas keamanan dalam satu kanal terpusat.

Konsep **Active Response human-in-the-loop** menjadi inti kebaruan penelitian ini. Berbeda dengan *Active Response* otomatis penuh yang langsung mengeksekusi tindakan (mis. blokir IP) untuk setiap *alert*, model *human-in-the-loop* menempatkan keputusan eksekusi tindakan kritis di tangan analis. Untuk *alert* dengan tingkat keparahan CRITICAL/HIGH, sistem mengirim notifikasi Telegram dengan dua tombol pilihan (“Isolasi File” atau “Abaikan”).

Eksekusi karantina berkas (quarantine-file) melalui Wazuh API baru terjadi setelah analisis menekan “Isolasi File”. Pendekatan ini menghindari risiko isolasi buta atas *false positive*, sebuah pertimbangan yang menjadi masukan langsung dari dosen pembimbing, sekaligus tetap mempertahankan kecepatan respons karena seluruh konteks (analisis, opsi tindakan) telah disiapkan otomatis oleh *pipeline*. Dengan adanya integrasi ini, sistem pertahanan siber tidak hanya bersifat reaktif, tetapi juga mampu memberikan respons terorkestrasi yang adaptif, cepat, namun tetap terkendali oleh manusia.

BAB III

METODOLOGI PENELITIAN

Bab III memuat metode yang digunakan dalam penelitian “Implementasi *Security Orchestration, Automation, and Response* (SOAR) Menggunakan *Stack* Sumber Terbuka untuk Deteksi Malware dan Phishing dengan *Active Response Human-in-the-Loop*”. Bagian metode penelitian membahas mengenai tempat dan waktu penelitian, data yang digunakan dalam penelitian, dan alur perancangan sistem yang dilakukan.

3.1 Waktu dan Tempat Penelitian

Penelitian ini dilakukan mulai dari bulan September 2025. Penelitian Tugas Akhir dilaksanakan di Kampus Bukit Universitas Udayana, Fakultas Teknik, Program Studi Teknologi Informasi yang beralamat di Jalan Kampus Bukit UNUD, Jimbaran, Kuta Selatan, Badung, Bali. Penelitian ini juga dilaksanakan di Perpustakaan Universitas Udayana yang beralamat di Jalan Kampus Bukit Udayana, Jimbaran, Kuta Selatan, Badung, Bali, dan rumah penulis yang berlokasi di Pasekan, Tabanan, Bali.

3.2 Sumber Data

Studi literatur dilakukan dengan cara mengumpulkan data dan informasi yang berkaitan dengan penerapan *tools* SIEM Wazuh, konsep SOAR, orkestrasi *workflow* dengan n8n, pengayaan *threat intelligence* VirusTotal, analisis AI lokal dengan Ollama, dan otomatisasi notifikasi serta respons melalui Telegram Bot. Data dan informasi ini didapatkan melalui jurnal dan sumber-sumber daring yang membantu pengembangan sistem ini. Beberapa sumber data dari studi literatur adalah sebagai berikut.

1. Jurnal-jurnal tentang penerapan SIEM Wazuh, konsep SOAR, deteksi *malware* dan *phishing*, *Active Response*, dan otomatisasi notifikasi Telegram Bot.
2. Dokumentasi resmi perangkat sumber terbuka yang digunakan, yaitu dokumentasi Wazuh, n8n, Ollama, VirusTotal API v3, Telegram Bot API, serta kerangka MITRE ATT&CK.
3. Data uji berupa *log* hasil simulasi aktivitas pada *endpoint*, termasuk berkas uji **EICAR** (*European Institute for Computer Antivirus Research*) sebagai pengganti *malware* yang aman, serta simulasi akses URL mencurigakan untuk pengujian deteksi *phishing*.

3.3 Instrumen Perancangan Sistem

Dalam membuat penelitian ini, penulis juga memerlukan beberapa instrumen pendukung untuk membantu pengembangan sistem. Instrumen pendukung tersebut dibagi ke dalam 2 komponen. Kedua komponen utama yang dibutuhkan untuk penelitian ini, yaitu perangkat keras (*hardware*) dan perangkat lunak (*software*).

3.3.1 Perangkat Keras (Hardware)

Perangkat keras yang diperlukan dalam penelitian ini terdiri dari satu laptop utama yang berperan sebagai **SOAR Server** (menjalankan Wazuh Manager, Indexer, Dashboard, n8n, dan Ollama) serta perangkat *endpoint* yang dipantau. Rincian spesifikasi perangkat keras yang digunakan penulis adalah sebagai berikut.

Tabel 3.1 Spesifikasi Perangkat Keras

Perangkat	Peran	Spesifikasi
Laptop Utama (ravi-zorin)	SOAR Server + Endpoint Agent #1	CPU Intel Core i5 1235U, RAM 16 GB, Memori 512 GB SSD, GPU MX 550, OS Ubuntu/Zorin Linux
ThinkPad X260 (rocky-server)	Endpoint Agent #2	CPU Intel Core i5, RAM ≥ 8 GB, OS Rocky Linux 9

Pemilihan dua *endpoint* dengan distribusi Linux berbeda (Debian-family dan RHEL-family) bertujuan untuk mendemonstrasikan kemampuan SOAR **lintas distribusi/multi-*endpoint***.

3.3.2 Perangkat Lunak (Software)

Perangkat lunak yang diperlukan dalam penelitian ini terdiri dari beberapa *tools* sumber terbuka. Rincian perangkat lunak yang digunakan penulis adalah sebagai berikut.

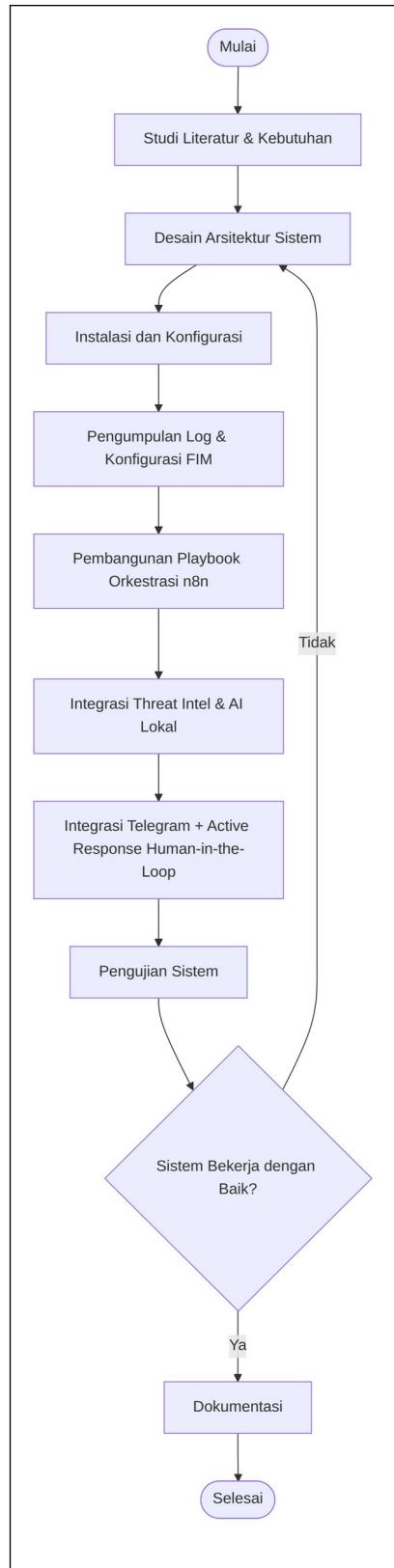
Tabel 3.2 Daftar Perangkat Lunak

Perangkat Lunak	Versi	Peran
Wazuh Manager	4.9.2	SIEM + korelasi <i>alert</i> + integratord
Wazuh Indexer	4.9.2	OpenSearch, penyimpanan & pencarian <i>log</i>
Wazuh Dashboard	4.9.2	Antarmuka visual (opsional)
Wazuh Agent	4.9.2	FIM + <i>endpoint monitoring</i>
Docker + Docker Compose	≥ 20.10 / v2	Kontainerisasi <i>stack</i>

Perangkat Lunak	Versi	Peran
		server
n8n	2.15.0	Mesin orkestrasi <i>workflow</i> (SOAR)
Ollama (llama3.2:3b)	latest	Inferensi LLM lokal untuk analisis
VirusTotal API	v3	<i>Threat intelligence</i> eksternal
Telegram Bot	-	Kanal notifikasi + tombol keputusan
Python	3.x	<i>Integration bridge + callback poller</i>
Tailscale	1.98.x	Mesh VPN untuk konektivitas antar-jaringan

3.4 Tahapan Penelitian

Tahapan penelitian menjelaskan langkah-langkah yang dilakukan dalam proses Implementasi Sistem SOAR Sumber Terbuka untuk Deteksi Malware dan Phishing dengan Active Response Human-in-the-Loop. Adapun alur penelitian ini terdiri dari beberapa tahap utama, seperti pada Gambar 3.1.



Gambar 3.1 Tahapan Penelitian

Gambar 3.1 di atas merupakan tahapan penelitian yang dilakukan selama pengerjaan tugas akhir ini. Tahapan ini dimulai dengan tahapan pertama, yaitu **studi literatur & kebutuhan**. Tahapan ini dilakukan dengan mengumpulkan berbagai referensi ilmiah dan dokumentasi teknis yang berkaitan dengan keamanan siber, SIEM Wazuh, konsep SOAR, orkestrasi n8n, deteksi malware/phishing, dan lainnya. Selain itu, dilakukan juga identifikasi terhadap kebutuhan perangkat keras dan perangkat lunak yang akan digunakan dalam penelitian, termasuk spesifikasi laptop utama sebagai SOAR Server dan *endpoint* yang akan dipantau.

Tahap kedua yaitu **desain arsitektur sistem** merupakan tahapan yang dilakukan dengan merancang arsitektur sistem SOAR berbasis Wazuh yang akan diintegrasikan dengan n8n, VirusTotal, Ollama, dan Telegram Bot. Rancangan tersebut meliputi struktur jaringan antar perangkat, alur komunikasi antar komponen sistem, desain integrasi *threat intelligence* dan AI dengan *log* Wazuh, serta rancangan alur komunikasi Telegram untuk *alert* dan *human-in-the-loop response*.

Tahap ketiga yaitu **instalasi dan konfigurasi**. Tahapan ini dilakukan dengan melakukan implementasi awal sistem di lingkungan *host* dan kontainer. Kegiatan utama meliputi instalasi *stack* Wazuh (Manager, Indexer, Dashboard) dan n8n menggunakan Docker Compose di laptop utama, instalasi Ollama sebagai *host service*, instalasi Wazuh Agent di *endpoint* (Ubuntu dan Rocky Linux) untuk mengirim *log* ke Wazuh Manager, konfigurasi jaringan (termasuk Tailscale mesh VPN agar *agent* tetap terhubung walau beda jaringan), dan uji koneksi antar mesin serta pendaftaran *agent* ke Wazuh Manager.

Tahap keempat yaitu **pengumpulan log & konfigurasi FIM**. Tahapan ini dilakukan dengan menerapkan konfigurasi *File Integrity Monitoring* pada *endpoint* untuk memantau lokasi-lokasi bernilai tinggi (sesuai pemetaan MITRE ATT&CK), serta melakukan simulasi aktivitas sistem untuk menghasilkan *log* yang mencakup aktivitas normal maupun aktivitas mencurigakan seperti *drop* berkas *malware* (EICAR) dan akses URL *phishing*.

Tahap kelima yaitu **pembangunan playbook orkestrasi n8n**. Tahapan ini dilakukan dengan membangun *workflow* di n8n yang menerima *alert* dari Wazuh melalui *webhook*, memfilter dan menormalisasi data, lalu mengoordinasikan langkah-langkah pengayaan dan klasifikasi *severity*.

Tahap keenam yaitu **integrasi threat intelligence & AI lokal**. Tahapan ini menghubungkan *playbook* dengan VirusTotal API untuk pemeriksaan reputasi *hash/URL*, dan dengan Ollama untuk menghasilkan analisis kontekstual berbahasa Indonesia yang menyesuaikan tingkat keparahan.

Tahap ketujuh yaitu **integrasi Telegram + Active Response human-in-the-loop**. Tahapan ini dilakukan dengan membuat Telegram Bot, membangun *workflow* notifikasi dengan *inline keyboard*, *callback poller* untuk meneruskan klik tombol ke n8n, serta *workflow* penanganan keputusan yang memanggil Wazuh API untuk mengeksekusi *quarantine-file* setelah disetujui analis.

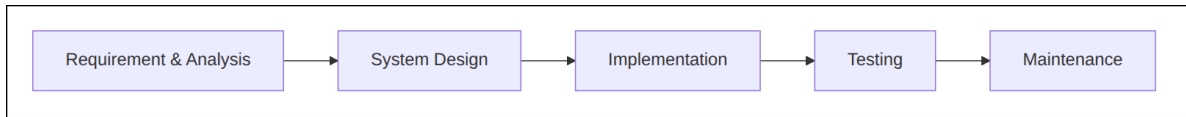
Tahap kedelapan yaitu **pengujian sistem**. Tahap ini dilakukan dengan serangkaian uji untuk memastikan fungsionalitasnya berjalan baik, seperti uji fungsionalitas, uji integrasi antarkomponen, uji performa notifikasi (MTTD/MTTR), dan uji respons (keberhasilan isolasi setelah persetujuan). Hasil pengujian dievaluasi untuk menilai apakah sistem sudah berjalan dengan baik; apabila belum, dilakukan pengulangan ke tahap desain arsitektur sistem.

Tahap kesembilan yaitu **dokumentasi**. Tahap ini dilakukan dengan mendokumentasikan seluruh proses penelitian ke dalam bentuk laporan tugas akhir dan presentasi. Dokumentasi ini mencakup deskripsi sistem dan konfigurasi, hasil pengujian dan analisis, serta saran untuk pengembangan selanjutnya.

3.5 Metode Pengembangan Sistem

Metode penelitian yang digunakan dalam penelitian ini adalah **metode pengembangan perangkat lunak Waterfall**. Model ini dipilih karena sesuai untuk proyek yang membutuhkan tahapan kerja yang sistematis dan berurutan, di mana setiap fase harus diselesaikan terlebih dahulu sebelum melanjutkan ke tahap berikutnya. Metode Waterfall memiliki lima tahap utama, yaitu:

1. Analisis Kebutuhan (*Requirement Analysis*)
2. Desain Sistem (*System Design*)
3. Implementasi (*Implementation*)
4. Pengujian (*Testing*)
5. Pemeliharaan (*Maintenance*)



Gambar 3.2 Tahapan Model Waterfall

Gambar 3.2 di atas merupakan model *waterfall* yang terdiri dari 5 tahapan yang berurutan dan saling berkaitan satu sama lainnya. Berikut ini adalah tahapan-tahapan model *waterfall* yang akan dijabarkan sesuai dengan proyek penelitian yang dilakukan.

3.5.1 Analisis Kebutuhan (Requirement Analysis)

Tahap ini bertujuan untuk mengidentifikasi kebutuhan fungsional dan non-fungsional dari sistem SOAR yang akan dikembangkan. Analisis dilakukan melalui studi literatur, observasi terhadap keterbatasan pendekatan deteksi-notifikasi manual, dan eksplorasi fitur yang disediakan oleh platform Wazuh dan n8n.

Kebutuhan fungsional sistem meliputi:

- Sistem mampu mengumpulkan dan menganalisis *log* keamanan dari *endpoint* menggunakan Wazuh.
- Sistem mampu mendeteksi *malware* melalui *File Integrity Monitoring* dan reputasi *hash* (VirusTotal).
- Sistem mampu mendeteksi *phishing* melalui analisis *log* akses URL.
- Sistem mampu mengorkestrasi *playbook* otomatis (n8n) untuk pengayaan *alert* dan klasifikasi *severity*.
- Sistem mampu menghasilkan analisis kontekstual menggunakan AI lokal (Ollama).
- Sistem mampu mengirimkan notifikasi *real-time* ke Telegram dengan tombol aksi untuk *alert* kritis.
- Sistem mampu mengeksekusi *Active Response* (isolasi berkas) **setelah** mendapat persetujuan analis (*human-in-the-loop*).
- Sistem menyediakan jejak audit (*audit trail*) untuk setiap insiden.

Kebutuhan non-fungsional sistem meliputi:

- Sistem dapat berjalan di lingkungan Linux (Ubuntu Server, Rocky Linux) dengan kontainerisasi Docker.

- Sistem dapat mendeteksi dan mengirim notifikasi dalam waktu yang relatif cepat (*real-time*, target 30-60 detik *end-to-end*).
- Sistem menjaga kedaulatan data, analisis AI berjalan lokal tanpa mengirim data ke pihak ketiga.
- Sistem tetap berfungsi walau *endpoint* berada di jaringan berbeda (melalui Tailscale).
- Sistem dibangun sepenuhnya dari perangkat sumber terbuka dengan biaya rendah.

3.5.2 Desain Sistem (System Design)

Tahap desain bertujuan untuk merancang arsitektur sistem berdasarkan kebutuhan yang telah diidentifikasi sebelumnya. Desain mencakup arsitektur logika sistem berlapis (*layered architecture*), desain alur data (*data flow*), perancangan *pipeline* deteksi dan klasifikasi *severity*, serta rancangan integrasi Telegram Bot dan *Active Response human-in-the-loop* yang akan dijelaskan pada sub bab 3.6, 3.7, dan 3.8.

3.5.3 Implementasi (Implementation)

Tahap implementasi merupakan proses menerjemahkan hasil desain ke dalam kode program dan sistem nyata. Langkah-langkah implementasi yang dilakukan adalah:

1. **Instalasi komponen sistem:** menjalankan *stack* Wazuh (Manager, Indexer, Dashboard) dan n8n melalui Docker Compose di laptop utama, instalasi Ollama sebagai *host service*, serta instalasi Wazuh Agent di setiap *endpoint*.
2. **Pengembangan *integration bridge*:** menyusun skrip *custom-n8n.py* yang dipanggil oleh *wazuh-integrator* untuk memfilter *noise* (berdasarkan *rule level* dan *path*), mengonversi *alert* Wazuh ke format JSON bersarang, lalu mengirimkannya ke *webhook* n8n yang sesuai (*malware* atau *phishing*).
3. **Pembangunan *playbook orkestrasi*:** membangun *workflow* “Deteksi Malware” dan “Deteksi Phishing” di n8n yang melakukan *filter*, ekstraksi, *scan* VirusTotal, klasifikasi *severity*, dan pembangunan *payload*.
4. **Integrasi AI lokal:** menghubungkan *playbook* dengan Ollama melalui `POST /api/generate` untuk menghasilkan analisis berbahasa Indonesia, lengkap dengan sanitasi karakter *markdown* agar kompatibel dengan Telegram.

5. **Integrasi Telegram bot:** membuat bot menggunakan BotFather, memperoleh API token, dan membangun notifikasi dengan *inline keyboard* (`callback_data = iso:<agentId> / ign:<agentId>`).
6. **Active Response script:** menyusun skrip quarantine-file di `/var/ossec/active-response/bin/` yang memindahkan berkas ke `/var/ossec/quarantine` dan mengubah hak akses menjadi `chmod 000`, serta `tg-callback-poller.py` yang melakukan *long-poll* `getUpdates` Telegram (aman di balik NAT) dan meneruskan klik tombol ke *webhook* `n8n` lokal.

3.5.4 Pengujian (Testing)

Tahap pengujian dilakukan untuk memastikan bahwa sistem berjalan sesuai dengan kebutuhan yang telah ditetapkan dan dapat mendeteksi serta menanggapi ancaman dengan baik. Indikator pengujian dijabarkan pada Tabel 3.5.

Tabel 3.5 Tabel Indikator Pengujian

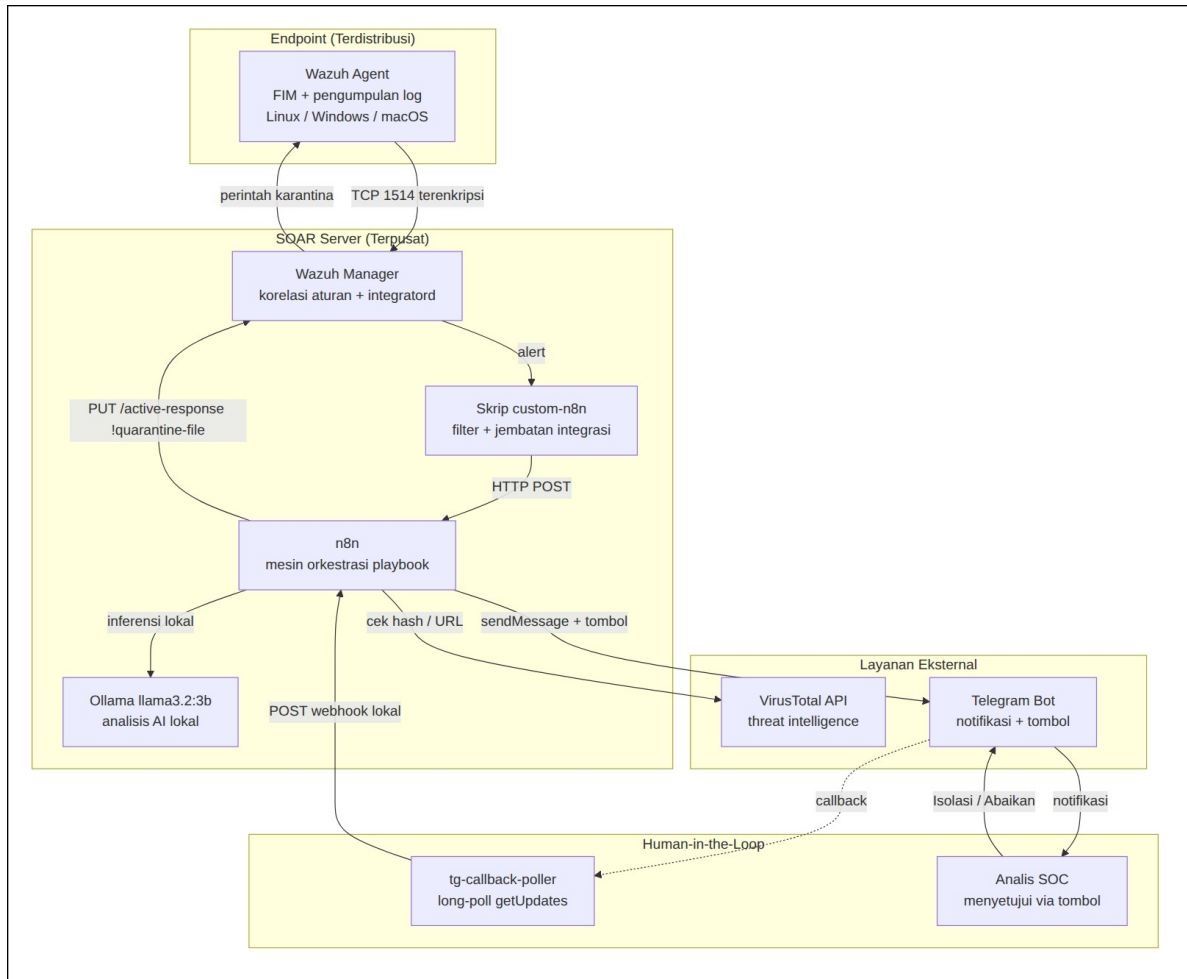
Jenis Pengujian	Tujuan	Indikator/Parameter
<i>Functional Testing</i>	Memastikan setiap fitur berfungsi sesuai kebutuhan	Deteksi <i>malware/phishing</i> , pengiriman notifikasi, <i>human-in-the-loop response</i>
<i>Performance Testing</i>	Mengukur waktu deteksi dan respons sistem	MTTD (<i>Mean Time to Detect</i>), MTTR (<i>Mean Time to Respond</i>)
<i>Integration Testing</i>	Menguji integrasi antar komponen (Wazuh, <code>n8n</code> , VirusTotal, Ollama, Telegram)	Keberhasilan pengiriman <i>log</i> , pemanggilan API, dan perintah otomatis
<i>Active Response Testing</i>	Menguji eksekusi isolasi berkas setelah persetujuan analis	Keberhasilan quarantine-file , status pesan <i>ter-update</i> (DIISOLASI/FALSE POSITIVE)
<i>Stress Testing</i>	Mengukur stabilitas sistem saat <i>log</i> berlimpah	Jumlah <i>event</i> /detik dan penggunaan CPU

3.5.5 Pemeliharaan (Maintenance)

Tahap terakhir adalah pemeliharaan sistem setelah implementasi dan pengujian berhasil dilakukan. Aktivitas pada tahap ini meliputi *monitoring* performa sistem selama periode uji operasional, perbaikan *bug*, penyetelan ambang klasifikasi *severity* dan *noise filter* apabila *false positive* yang dihasilkan tinggi, pembaruan *ruleset* Wazuh secara berkala agar dapat menangani ancaman baru, pemeliharaan token Telegram/VirusTotal/Wazuh, serta pencadangan (*backup*) definisi *workflow* n8n secara rutin. Tahap pemeliharaan menjamin sistem tetap adaptif terhadap ancaman baru serta siap untuk dikembangkan lebih lanjut.

3.6 Desain Arsitektur Sistem

Desain arsitektur sistem ini menggambarkan hubungan dan alur komunikasi antar komponen utama yang membentuk sistem SOAR sumber terbuka berbasis Wazuh, n8n, VirusTotal, Ollama, dan Telegram Bot. Berikut ini adalah gambaran desain arsitektur sistem yang telah dirancang, seperti pada Gambar 3.3.



Gambar 3.3 Desain Arsitektur Sistem SOAR

Sistem dirancang dengan **lima lapisan (layer)** yang terpisah dengan tanggung jawab yang jelas, sehingga setiap komponen memiliki batasan peran yang tegas (*high cohesion, low coupling*).

Layer 1: Deteksi (Distributed). Berada di setiap *endpoint*, dijalankan oleh **Wazuh Agent**. Komponen ini melakukan *File Integrity Monitoring* secara *realtime* (melalui inotify pada Linux), pengumpulan *log*, dan pemantauan aktivitas proses/jaringan. *Footprint*-nya sangat ringan (~50 MB RAM, <1% CPU). Keluaran berupa *event* dikirim ke Wazuh Manager melalui TCP 1514 terenkripsi.

Layer 2: Agregasi & Korelasi (Centralized). Berada di SOAR Server (kontainer Wazuh Manager). Komponen *wazuh-remoted* menerima koneksi *agent*, *wazuh-analysisd* melakukan pencocokan aturan dan *decoding*, dan *wazuh-integratord* memicu skrip integrasi berdasarkan

whitelist rule_id. Keluaran berupa *alert* terstruktur yang diteruskan ke skrip integrasi custom-n8n.

Layer 3: Orkestrasi (Centralized). Berada di SOAR Server (kontainer n8n). Mesin *workflow* n8n menerima *alert* melalui *webhook*, memfilter dan menormalisasi data, mengoordinasikan pengayaan (VirusTotal), melakukan klasifikasi *severity* dari berbagai sumber (*rule_level* + skor VT), lalu merutekan ke aksi respons atau notifikasi.

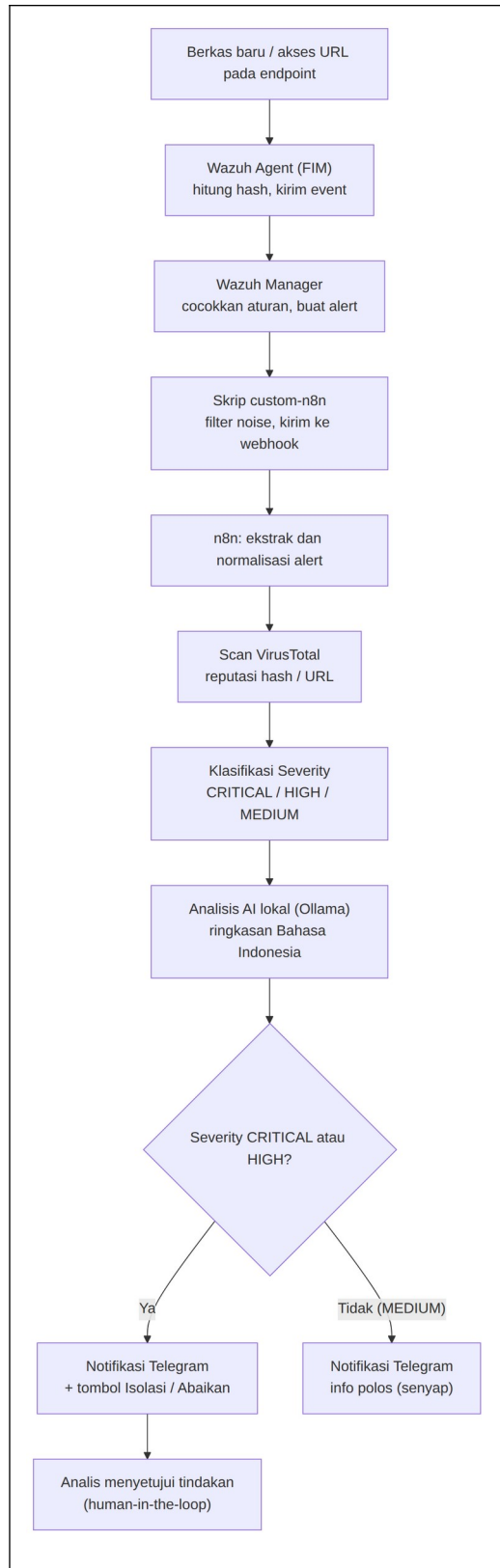
Layer 4: Pengayaan & AI (Centralized). Terdiri dari **VirusTotal API** (intel ancaman eksternal, reputasi *hash*/URL terhadap 60+ mesin antivirus) dan **Ollama** (LLM lokal untuk analisis kontekstual berbahasa Indonesia, dengan *prompt* yang menyesuaikan *severity*). Inferensi AI berjalan **lokal** sehingga data tidak keluar dari server (kedaulatan data).

Layer 5: Eksekusi Respons (Distributed, human-approved). Berada di setiap *endpoint* (Wazuh agent dengan skrip *Active Response* kustom quarantine-file). Model yang dipakai adalah **human-in-the-loop**: *Active Response* **tidak dipicu otomatis** oleh aturan, melainkan hanya berjalan setelah analis menyetujui melalui tombol Telegram. Untuk *alert* CRITICAL/HIGH, n8n mengirim Telegram + *inline keyboard*; jika analis memilih “Isolasi File”, *handler* memanggil Wazuh API PUT /active-response dengan perintah !quarantine-file, dan *agent* memindahkan berkas ke karantina serta mengubah hak akses menjadi chmod 000. Jika analis memilih “Abaikan”, pesan ditandai sebagai FALSE POSITIVE tanpa ada aksi.

Sistem terdiri dari dua lingkungan utama, yaitu **SOAR Server** (laptop utama yang menjalankan Wazuh Manager/Indexer/Dashboard, n8n, dan Ollama) dan **Endpoint** terdistribusi (laptop Ubuntu dan ThinkPad Rocky Linux). Konektivitas antar-jaringan dijaga oleh **Tailscale mesh VPN** sehingga pipeline tetap berjalan walaupun *endpoint* berpindah jaringan, tanpa perlu *port forwarding*.

3.7 Perancangan Pipeline Deteksi dan Klasifikasi Severity

Bagian ini menjelaskan proses *pipeline* deteksi *end-to-end* mulai dari terjadinya *event* pada *endpoint* hingga pengiriman notifikasi. *Pipeline* dirancang sebagai serangkaian langkah terorkestrasi yang dapat ditelusuri (*traceable*), seperti pada Gambar 3.4.



Gambar 3.4 Alur Pipeline Deteksi End-to-End

Alur kerja *pipeline* deteksi *malware* adalah sebagai berikut.

1. **Deteksi di Endpoint:** pengguna atau penyerang membuat berkas baru (mis. ~/Downloads/malware.exe). Daemon wazuh-syscheckd menangkap *event* inotify, menghitung *hash* (SHA-256/SHA-1/MD5), dan mengirim *event* ke Manager melalui TCP 1514 terenkripsi (waktu <100 ms).
2. **Agregasi di Manager:** wazuh-analysisd mendekode *event* dan mencocokkannya dengan aturan (mis. *rule* 554 “File added to the system”), lalu menghasilkan *alert* JSON.
3. **Pemicu Integrasi:** wazuh-integratord memanggil skrip custom-n8n.py sesuai konfigurasi <integration> (berdasarkan *whitelist rule_id*).
4. **Integration Bridge (custom-n8n.py):** skrip melakukan penyaringan berlapis: (a) *threshold rule_level* (≥ 5), (b) *path-based noise filter* (mengabaikan *path* sah seperti /tmp/runc-process*, /var/cache/), lalu (c) deteksi jenis *event*, URL diarahkan ke *webhook phishing*, *hash* ke *webhook malware*. *Payload* dikonversi ke JSON bersarang dan dikirim melalui HTTP POST.
5. **Orkestrasi di n8n:** *workflow* memfilter *alert* (memeriksa keberadaan *hash*), menormalisasi *field* (Ekstrak Alert), lalu melakukan *scan* VirusTotal (GET /files/{sha256}). Apabila *hash* tidak dikenal (404), *workflow* tetap berlanjut dengan *malicious=0* (neverError: true).
6. **Klasifikasi Severity:** pada *node* Rangkum Hasil, tingkat keparahan ditentukan berdasarkan kombinasi skor VirusTotal dan *rule_level* sesuai Tabel 3.3.

Tabel 3.3 Aturan Klasifikasi Severity

Severity	Pemicu	Notifikasi Telegram	Tombol Aksi (Human-in-the-Loop)
CRITICAL	malicious ≥ 20 ATAU <i>rule_level</i> ≥ 12	KRITIS, <i>sound on</i>	Ya (tombol Isolasi/Abaikan)
HIGH	malicious ≥ 5 ATAU	TINGGI, <i>sound on</i>	Ya (tombol Isolasi/Abaikan)

Severity	Pemicu	Notifikasi Telegram	Tombol Aksi (Human-in-the-Loop)
	<code>rule_level ≥ 7</code>		
MEDIUM	(selain di atas)	SEDANG, <i>silent</i>	Tidak (info polos, tanpa tombol)

Nilai `should_active_response` ditetapkan TRUE untuk CRITICAL/HIGH dan FALSE untuk MEDIUM. Variabel ini **tidak** memicu isolasi otomatis; ia hanya menentukan apakah notifikasi Telegram diberi *inline keyboard* (tombol aksi).

7. **Pengayaan AI:** *node* Build Payload menyusun *prompt* yang menyesuaikan *severity* (mis. untuk CRITICAL: “berikan rekomendasi *immediate response*, isolasi sistem, eradikasi”), lalu memanggil Ollama (llama3.2:3b). Respons dibersihkan dari *tag* berpikir dan karakter *markdown* agar tidak menyebabkan *error* pada Telegram.
8. **Pengiriman Notifikasi:** untuk CRITICAL/HIGH, *node* “Send Telegram Alert” mengirim pesan dengan dua tombol; untuk MEDIUM, “Send Telegram Info” mengirim pesan polos dan senyap. Latensi *end-to-end* (deteksi → notifikasi) yang teramati berkisar **30-60 detik**, dengan *bottleneck* utama pada inferensi Ollama (CPU-bound, 15-50 s) dan panggilan VirusTotal (5-15 s).

Pemetaan lokasi pemantauan FIM terhadap kerangka MITRE ATT&CK ditunjukkan pada Tabel 3.4.

Tabel 3.4 Pemetaan Lokasi FIM terhadap MITRE ATT&CK

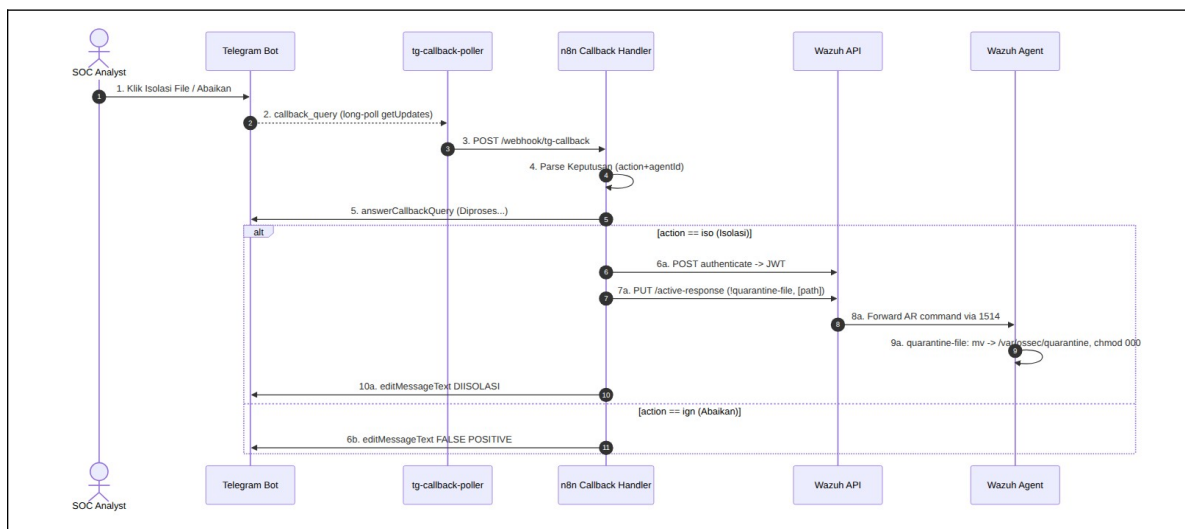
Lokasi Dipantau	Teknik MITRE	Deskripsi
<code>/etc/cron.*</code> , <code>/var/spool/cron</code>	T1053.003	<i>Scheduled Task: Cron</i>
<code>/etc/systemd/system</code>	T1543.002	<i>Create/Modify System Process: Systemd</i>
<code>~/.ssh/</code> , <code>/root/.ssh/</code>	T1098.004	<i>Account Manipulation: SSH Keys</i>
<code>/usr/local/bin</code>	T1554	<i>Compromise Client Software Binary</i>
<code>~/Downloads</code> , <code>~/Desktop</code>	T1566	<i>Phishing</i> (akses awal)

3.8 Desain Active Response Human-in-the-Loop dan Integrasi Telegram Bot

Integrasi Telegram Bot merupakan bagian dari sistem yang berfungsi untuk menghubungkan hasil deteksi keamanan dari Wazuh SIEM dan lapisan orkestrasi kepada administrator jaringan secara *real-time*. Komponen ini berperan sebagai **antarmuka**

komunikasi dua arah antara sistem keamanan dan analis, sehingga respons terhadap insiden dapat dilakukan dengan cepat tanpa harus mengakses *dashboard* Wazuh. Tujuan utama integrasi ini adalah meningkatkan kecepatan respons insiden, menyediakan jalur komunikasi dua arah, dan membuat sistem menjadi interaktif sehingga analis dapat memberikan perintah langsung seperti mengisolasi berkas atau mengabaikan *alert* yang dianggap *false positive*.

Inti dari desain ini adalah model **Active Response *human-in-the-loop***. Berbeda dengan desain *auto-Active Response* (yang langsung mengeksekusi tindakan untuk setiap *alert*), pada desain ini eksekusi tindakan kritis baru terjadi setelah analis menyetujui. Alur lengkap ditunjukkan pada Gambar 3.5 dan Gambar 3.6.



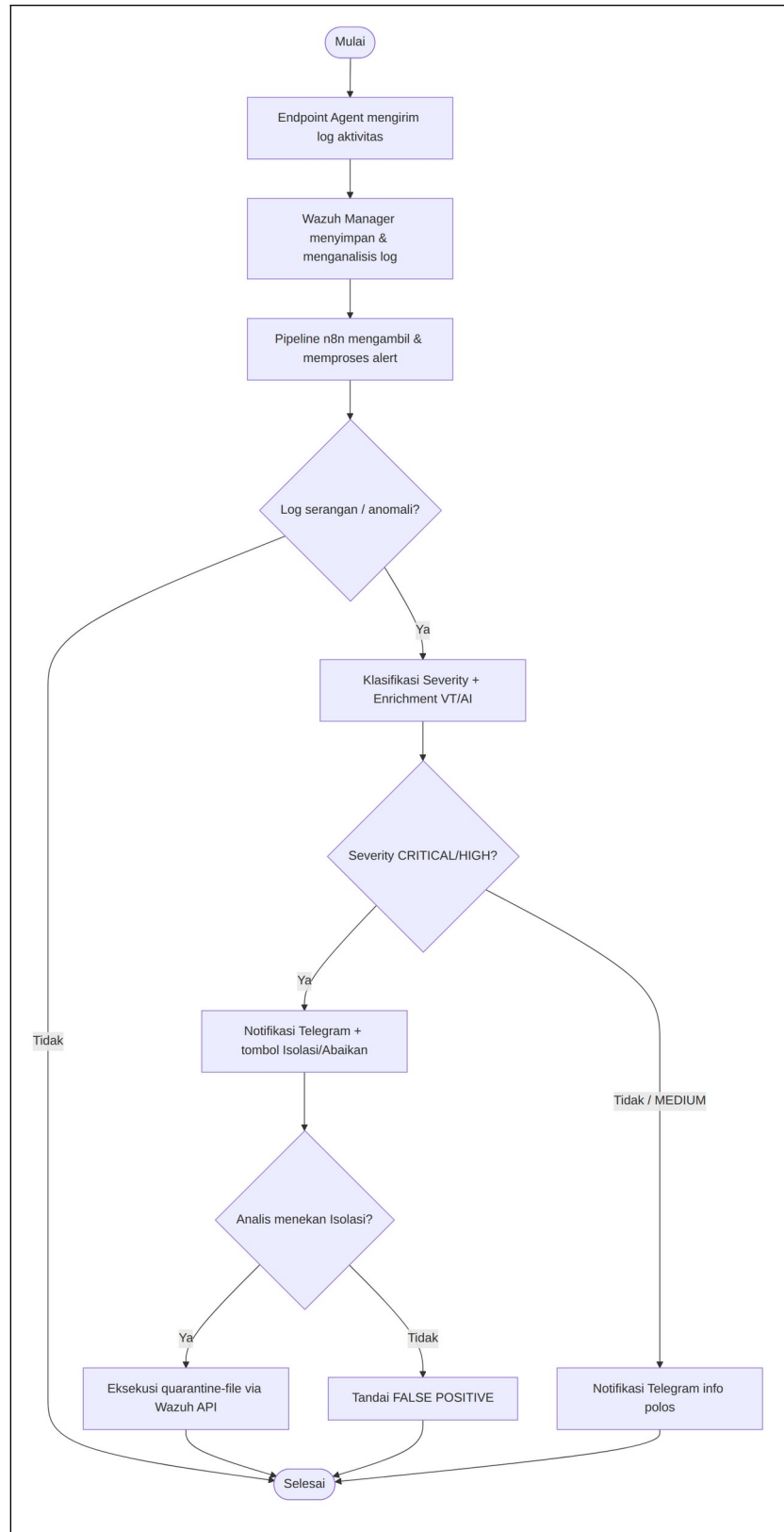
Gambar 3.5 Sequence Active Response Human-in-the-Loop

Tahapan desain *Active Response human-in-the-loop* dijelaskan sebagai berikut.

1. **Notifikasi dengan tombol:** untuk *alert* CRITICAL/HIGH, *workflow* “Deteksi Malware” mengirim notifikasi Telegram yang dilengkapi *inline keyboard* dua tombol, dengan `callback_data = iso:<agentId>` (Isolasi File) atau `ign:<agentId>` (Abaikan). Untuk *alert* MEDIUM, pesan dikirim polos tanpa tombol.
2. **Penerusan klik tombol (tg-callback-poller):** n8n Telegram Trigger berbasis *webhook* publik, sehingga tidak dapat dipakai di balik NAT/Tailscale yang tidak memiliki URL publik. Untuk mengatasinya, dibuat *poller* yang melakukan *long-poll* `getUpdates` (koneksi keluar saja) dan meneruskan *update* `callback_query` ke *webhook* n8n **lokal**

/webhook/tg-callback. *Poller* menjalankan `deleteWebhook` saat *start* agar tidak terjadi konflik `getUpdates` (409). Pendekatan ini menjaga agar tidak ada *port* `n8n` yang terekspos ke internet.

3. **Penanganan keputusan:** *workflow* “Telegram Callback Handler” menerima *callback*, mem-*parse* keputusan (*action* + *agentId* dari *callback_data*, serta *path/hash* dari teks pesan), lalu memanggil `answerCallbackQuery`.
4. **Eksekusi isolasi (jika disetujui):** apabila analis memilih “Isolasi File”, *handler* melakukan autentikasi ke Wazuh API (`POST /security/user/authenticate` → JWT), lalu memanggil `PUT /active-response?agents_list=<id>` dengan *body* `{"command": "!quarantine-file", "arguments": ["<path>"]}`. Wazuh Manager meneruskan perintah ke *agent* melalui TCP 1514, dan `wazuh-execd` menjalankan skrip `quarantine-file` yang memindahkan berkas ke `/var/ossec/quarantine/<nama>.<timestamp>.quarantined` dan menetapkan `chmod 000`. Pesan Telegram diperbarui menjadi “DIISOLASI”.
5. **Penanganan false positive:** apabila analis memilih “Abaikan”, pesan diperbarui menjadi “FALSE POSITIVE” tanpa tindakan apa pun pada berkas.



Gambar 3.6 Desain Integrasi Telegram Bot

Catatan penting pada desain ini adalah bahwa *node* firewall-drop lama (auto-Active Response berbasis pemblokiran IP otomatis) telah **dihapus** dari *workflow*. Alur aktif sepenuhnya memakai quarantine-file melalui persetujuan analis. Setiap insiden meninggalkan jejak audit lengkap di enam lokasi: *log agent* Wazuh (ossec.log), *log alert* Manager (alerts.log), *log Active Response* (active-responses.log), *log* integrasi (integrations.log), *log* eksekusi n8n (di antarmuka n8n), dan riwayat pesan Telegram. Jejak audit ini menjadi bukti verifikasi bahwa sistem berfungsi dan menjadi dasar analisis forensik.

DAFTAR PUSTAKA

- Ariyanto, Y., Watequlis Syaifudin, Y., Hasyim Ratsanjani, M., Ridho Muladawila, A., Fatmawati, T., Yoga Saputra, P., & Setiadi, C. (2025). *Cyber Threat Detection and Automated Response Using Wazuh and Telegram API*. *Matrik*, 25(1), 173-188.
<https://doi.org/10.30812/matrik.v25i1.5610>
- Delvita aulia artika, D., Rumahorbo, D. R., haikal, M. haikal A.-M., & Kiswanto, D. (2025). *Implementasi Sistem Keamanan Website dengan Analisis Log dan Deteksi Aktivitas Anomali Menggunakan Isolation Forest*. *Jurnal Informatika Dan Teknik Elektro Terapan*, 13(3S1). <https://doi.org/10.23960/jitet.v13i3S1.8133>
- Dwi Prasetyo, O., Hari Trisnawan, P., & Bhawiyuga, A. (2023). *Uji Kinerja Host-Based Intrusion Detection System WAZUH terhadap Serangan Brute Force dan DoS* (Vol. 7, Issue 6). <http://j-ptiik.ub.ac.id>
- Dyan Heluka, H., & Sulistyo, W. (n.d.). *Perancangan Dan Implementasi Security Information and Event Management (SIEM) pada Layanan Virtual Server*.
- Farrel, F. I. F., Is Mardianto, S.Si, M.Kom, & Ir. Adrian Sjamsul Qamar, MTI. (2024). *Implementation of Security Information & Event Management (SIEM) Wazuh with Active Response and Telegram Notification for Mitigating Brute Force Attacks on The GT-I2TI USAKTI Information System*. *Intelmatiks*, 4(1), 1-7.
<https://doi.org/10.25105/itm.v4i1.18529>
- Gartner. (2023). *Market Guide for Security Orchestration, Automation and Response (SOAR) Solutions*. Gartner Research.
- Gede Parama Antara, & Ika Dyah Agustia Rachmawati. (2024). *Implementasi dan Analisis Wazuh Sebagai Intrusion Detection System (IDS) dan Platform Monitoring*. *Jurnal Informasi, Sains Dan Teknologi*, 7(2), 290-303.
<https://doi.org/10.55606/isaintek.v7i2.301>
- MITRE. (2024). *MITRE ATT&CK Framework*. <https://attack.mitre.org/>
- n8n GmbH. (2024). *n8n Workflow Automation Documentation*. <https://docs.n8n.io/>

Ollama. (2024). *Ollama API Documentation*.

<https://github.com/ollama/ollama/blob/main/docs/api.md>

Puteri, et al. (2024). *Konsep dan Penerapan Keamanan Siber Berbasis NIST Cybersecurity Framework*. (Jurnal Keamanan Siber).

Saputra, F. A., et al. (2024). *Penerapan Wazuh sebagai SIEM Terintegrasi Telegram Bot untuk Deteksi dan Visualisasi Log Keamanan Real-time (Studi Kasus PeTIK Jombang)*.

Telegram. (2024). *Telegram Bot API*. <https://core.telegram.org/bots/api>

VirusTotal. (2024). *VirusTotal API v3 Reference*. <https://docs.virustotal.com/reference/overview>

Wazuh Inc. (2024). *Wazuh Documentation v4.9*. <https://documentation.wazuh.com/4.9/>